

---

# Organization

Some of the most important changes in ES6 involve improved support for the patterns we already commonly use to organize JavaScript functionality. This chapter will explore Iterators, Generators, Modules, and Classes.

## Iterators

An *iterator* is a structured pattern for pulling information from a source in one-at-a-time fashion. This pattern has been around programming for a long time. And to be sure, JS developers have been ad hoc designing and implementing iterators in JS programs since before anyone can remember, so it's not at all a new topic.

What ES6 has done is introduce an implicit standardized interface for iterators. Many of the built in data structures in JavaScript will now expose an iterator implementing this standard. And you can also construct your own iterators adhering to the same standard, for maximal interoperability.

Iterators are a way of organizing ordered, sequential, pull-based consumption of data.

For example, you may implement a utility that produces a new unique identifier each time it's requested. Or you may produce an infinite series of values that rotate through a fixed list, in round-robin fashion. Or you could attach an iterator to a database query result to pull out new rows one at a time.

Though not as common a usage of iterators to this point in JS, iterators can also be thought of as controlling behavior one step at a time. This can be illustrated quite clearly when considering generators (see “Generators” later in this chapter), though you can certainly do the same without generators.

## Interfaces

At the time of this writing, ES6 section 25.1.1.2 (<https://people.mozilla.org/~jorendorff/es6-draft.html#sec-iterator-interface>) details the `Iterator` interface as having the following requirement:

```
Iterator [required]
  next() {method}: retrieves next IteratorResult
```

There are two optional members which some iterators are extended with:

```
Iterator [optional]
  return() {method}: stops iterator and returns IteratorResult
  throw() {method}: signals error and returns IteratorResult
```

The `IteratorResult` interface is specified as:

```
IteratorResult
  value {property}: current iteration value or final return value
    (optional if `undefined`)
  done {property}: boolean, indicates completion status
```



I call these interfaces implicit not because they're not explicitly called out in the specification — they are! — but because they're not exposed as direct objects accessible to code. JavaScript does not, in ES6, support any notion of “interfaces”, so adherence for your own code is purely conventional. However, wherever JS expects an iterator — a `for..of` loop, for instance — what you provide must adhere to these interfaces or the code will fail.

There's also an `Iterable` interface, which describes objects that must be able to produce iterators:

```
Iterable
  @@iterator() {method}: produces an Iterator
```

If you recall from “Built-in Symbols” in Chapter 2, `@@iterator` is the special built-in symbol representing the method that can produce iterator(s) for the object.

### IteratorResult

The `IteratorResult` interface specifies that the return value from any iterator operation will be an object of the form:

```
{ value: .. , done: true / false }
```

Built-in iterators will always return values of this form, but it is of course allowed that more properties be present on the return value, as necessary.

For example, a custom iterator may add additional metadata to the result object, like where the data came from, how long it took to retrieve, cache expiration length, frequency for the appropriate next request, etc.



Technically, `value` is optional if it would otherwise be considered absent or unset, such as in the case of the value of `undefined`. Since accessing `res.value` will produce `undefined` whether it's present with that value or absent entirely, the presence/absence of the property is more an implementation detail and/or an optimization, rather than a functional issue.

## `next()` Iteration

Let's look at an array, which is an iterable, and the iterator it can produce to consume its values:

```
var arr = [1,2,3];

var it = arr[Symbol.iterator]();

it.next();    // { value: 1, done: false }
it.next();    // { value: 2, done: false }
it.next();    // { value: 3, done: false }

it.next();    // { value: undefined, done: true }
```

Each time the method located at `Symbol.iterator` (see Chapter 2 and 7) is invoked on this `arr` value, it will produce a new fresh iterator. Most structures will do the same, including all the built-in data structures in JS.

However, it *is* possible to conceive of a structure which could only produce a single iterator (singleton pattern), or perhaps only allow one unique iterator at a time, requiring the current one to be

You'll notice that the `it` iterator in the previous snippet doesn't report `done: true` when you received the 3 value. You have to call `next()` again, in essence going beyond the end of the array's values, to get the completed signal `done: true`. It may not be clear why until later in this section, but that design decision will typically be considered a best practice.

Primitive string values are also iterables by default:

```
var greeting = "hello world";

var it = greeting[Symbol.iterator]();

it.next();    // { value: "h", done: false }
```

```
it.next();    // { value: "e", done: false }
..
```

ES6 also includes several new data structures, called Collections (see Chapter 5). These collections are not only iterables themselves, but they also provide API method(s) to generate an iterator, such as:

```
var m = new Map();
m.set( "foo", 42 );
m.set( { cool: true }, "hello world" );

var it1 = m[Symbol.iterator]();
var it2 = m.entries();

it1.next();    // { value: [ "foo", 42 ], done: false }
it2.next();    // { value: [ "foo", 42 ], done: false }
..
```

The `next(..)` method of an iterator can optionally take one or more arguments. The built-in iterators mostly do not exercise this capability, though a generator’s iterator definitely does (see “Generators” later in this chapter).

By general convention, including all the built-in iterators, calling `next(..)` on an iterator that’s already been exhausted is not an error, but will simply continue to return the result `{ value: undefined, done: true }`.

## Optional: `return(..)` and `throw(..)`

The optional methods on the iterator interface — `return(..)` and `throw(..)` — are not implemented on most of the built-in iterators. However, they definitely do mean something in the context of generators, so see “Generators” for more specific information.

`return(..)` is defined as sending a signal to an iterator that the consuming code is complete and will not be pulling any more values from it. This signal can be used to notify the producer (the iterator responding to `next(..)` calls) to perform any cleanup it may need to do, such as releasing/closing network, database, or file handle resources, etc.

If an iterator has a `return(..)` present and any condition occurs which can automatically be interpreted as abnormal or early termination of consuming the iterator, `return(..)` will automatically be called. You can call `return(..)` manually as well.

`return(..)` will return an `IteratorResult` object just like `next(..)` does. In general, the optional value you send to `return(..)` would be sent back as `value` in this `IteratorResult`, though there are nuanced cases where that might not be true.

`throw(..)` is used to signal an exception/error to an iterator, which possibly may be used differently by the iterator than the completion signal implied by `return(..)`. It does not necessarily imply a complete stop of the iterator as `return(..)` generally does.

For example, with generator iterators, `throw(..)` actually injects a thrown exception into the generator's paused execution context, which can be caught with a `try..catch`. An uncaught `throw(..)` exception would end up abnormally aborting the generator's iterator.



By general convention, an iterator should not produce any more results after having called `return(..)` or `throw(..)`.

## Iterator Loop

As we covered in the "for..of" section in Chapter 2, the ES6 `for..of` loop directly consumes a conforming iterable.

If an iterator is also an iterable, it can be used directly with the `for..of` loop. You make an iterator an iterable by giving it a `Symbol.iterator` method that simply returns the iterator itself:

```
var it = {  
  // make the `it` iterator an iterable  
  [Symbol.iterator]() { return this; },  
  
  next() { .. },  
  ..  
};  
  
it[Symbol.iterator]() === it;    // true
```

Now we can consume the `it` iterator with a `for..of` loop:

```
for (var v of it) {  
  console.log( v );  
}
```

To fully understand how such a loop works, let's consider this more manual version of the previous snippet's loop:

```
for (var v, res; !(res = it.next()) && !res.done; ) {  
  v = res.value;  
  console.log( v );  
}
```

If you look closely, you'll see that `it.next()` is called before each iteration, and then `res.done` is consulted. If `res.done` is `false`, the iteration doesn't occur.

Recall earlier that we suggested iterators should in general not return `done: true` along with the final intended value from the iterator. Here you can see why.

If an iterator returned `{ done: true, value: 42 }`, the `for..of` loop would completely discard the 42 value and it'd be unavailable. For this reason — to assume that your iterator may be consumed by such patterns as the `for..of` loop or its manual equivalent — you should probably wait to return `done: true` for signaling completion until after you've already returned all relevant iteration values.



You can of course intentionally design your iterator to return some relevant value at the same time as returning `done: true`. Don't do this unless you've documented that as the case, and thus implicitly forced consumers of your iterator to use a different pattern for iteration than is implied by `for..of` or its manual equivalent we depicted.

## Custom Iterators

In addition to the standard built-in iterators, you can make your own! All it takes to make them interoperate with ES6's consumption facilities (e.g., the `for..of` loop and the `... operator`) is to adhere to the proper interface(s).

Let's try constructing an iterator that produces the infinite series of numbers in the Fibonacci sequence:

```
var Fib = {
  [Symbol.iterator]() {
    var n1 = 1, n2 = 1;

    return {
      // make the iterator an iterable
      [Symbol.iterator]() { return this; },

      next() {
        var current = n2;
        n2 = n1;
        n1 = n1 + current;
        return { value: current, done: false };
      },

      return(v) {
        console.log(
          "Fibonacci sequence abandoned."
        );
        return { value: v, done: true };
      }
    };
  }
};
```